# DD: BIL-02-ADJ-01 — Invoice Adjustments

Developer implementation guide: `DD_BIL-02-ADJ-01-Invoice_Adjustments-DEV_IMPL_GUIDE-v1.3.md`.

## 1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_BIL-02-ADJ-01-Invoice_Adjustments-v1.3.md`.

**Verdict:** ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.3 Satellite of **BIL-02 Invoicing (Core)**. Owns the admin workflow for proposing, approving, and applying invoice adjustments — credit notes (money returned to the customer) and debit notes (additional money owed by the customer). Invokes the generator classes that live in BIL-02-GEN-01 Invoice Generation. Coordinates the approval gate via Camunda per the operator's configured workflow. **Sibling DDs**: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery. **Version history**: see `CHANGELOG_BIL-02-ADJ-01-Invoice_Adjustments.md` in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

## 2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

## 3. Runtime Metadata

| Item | Value |
|---|---|
| Source file | `/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-ADJ-01-Invoice_Adjustments-v1.3.md` |
| Version | `v1.3` |
| DD type | `module contract` |
| BPMN/process keys | Not explicitly defined |
| Drools packages | Not explicitly defined |

## 4. Runtime Contracts To Implement

### 4.1 APIs

- `GET /api/adjustments`
- `GET /api/adjustments/reporting/by-customer`
- `GET /api/adjustments/reporting/by-reason`
- `GET /api/adjustments/reporting/time-to-apply`
- `GET /api/adjustments/{id}`
- `GET /api/credit-notes/{id}`

- `GET /api/credit-notes/{id}/pdf`
- `POST /api/adjustments`
- `POST /api/adjustments/{id}/approve`
- `POST /api/adjustments/{id}/cancel`
- `POST /api/adjustments/{id}/override-limit`
- `POST /api/adjustments/{id}/reject`
- `POST /api/adjustments/{id}/request-revision`
- `POST /api/adjustments/{id}/retry-application`

### 4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

### 4.3 Worker Topics

- `billing-invoicing`
- `billing-invoicing-adjustments`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

### 4.4 Events

- `InvoiceGenerationContext`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

## 5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.

10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

# 6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

| Concern | Recommendation |
|---|---|
| API/runtime | Run inside the owning module deployment or existing workflow API pods. |
| Workers | Consolidate low-volume topics into shared worker apps; keep per-topic metrics. |
| Database | Use the owning module PostgreSQL schema and migration pipeline. |
| Kafka | Use the foundation Kafka/MSK setup and transactional outbox. |
| Camunda | Deploy BPMN files through the workflow deployment pipeline. |
| Drools | Deploy KJAR/rule artifacts through the approved rules pipeline. |
| Secrets | Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA. |
| Observability | Alert on stuck operations, failed workers, outbox backlog, and external dependency failures. |

# 7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

**Verdict:** ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.3

Satellite of **BIL-02 Invoicing (Core)**. Owns the admin workflow for proposing, approving, and applying invoice adjustments — credit notes (money returned to the customer) and debit notes (additional money owed by the customer). Invokes the generator classes that live in BIL-02-GEN-01 Invoice Generation. Coordinates the approval gate via Camunda per the operator's configured workflow.

**Sibling DDs**: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery.

**Version history**: see `CHANGELOG_BIL-02-ADJ-01-Invoice_Adjustments.md` in the same directory.

# Glossary

| Term | Meaning |
|---|---|
| **Adjustment** | A corrective entry posted against a previously-issued invoice. Two kinds: credit note and debit note. |
| **Credit note** | A document that credits the customer — money goes back to them. For POSTPAID, reduces outstanding balance on the parent invoice (or future invoices). For PREPAID, credits the customer's wallet (the wallet specified at the adjustment time — typically the one debited by the original charge). Generated by `CreditNoteGenerator` (owned by BIL-02-GEN-01 Invoice Generation). |
| **Debit note** | A document that debits the customer — money is added to what they owe. For POSTPAID, increases outstanding balance (treated as an additional charge). For PREPAID, debits the customer's wallet (if balance allows; otherwise the operator's configured behavior — see rules). Generated by `DebitNoteGenerator` (owned by BIL-02-GEN-01). |
| **Parent invoice** | The original cycle invoice, one-off invoice, or pro forma that an adjustment is correcting. Linked via `original_invoice_id` on the credit/debit note. |
| **Adjustment request** | An admin-submitted proposal for an adjustment. Holds the proposed amount, reason, scope (which parent invoice, which line items if partial), and current approval state. Persisted in `adjustment_request` table. Once approved, the system calls the appropriate generator. |
| **Approval workflow** | A Camunda BPMN process that gates an adjustment request before it's applied. Per-operator configuration: some operators may auto-approve below a threshold, single-step approve above, multi-step approve for very large amounts; others may have no approval (zero-step). Defined per the SUB-WF-FRAMEWORK-01 Subscription Workflow Framework approach. |

| Term | Meaning |
|---|---|
| **Adjustment scope** | Three modes deciding how the adjustment amount is determined and how it ties back to the parent invoice. **FULL**: covers the entire parent invoice. **LINE**: covers one or more specific line items of the parent invoice — the admin selects which lines, and may optionally reduce the amount per line (partial credit for proration). **AMOUNT**: free-form value not tied to any parent line — used for goodwill, settlements, or compensation that doesn't map to a specific service charge. The three modes have distinct business use cases — see "Scopes and when to use them" below. |
| **Reason code** | A catalog value identifying the business reason for the adjustment. Operators configure their own reason code catalog; ADJ-01 enforces that every adjustment carries a reason code for audit and reporting. |
| **Limits policy** | Per-operator rules constraining adjustments: maximum credit amount per request, maximum credit amount per customer per period, debit cannot push wallet below allowed minimum, cannot adjust signed tax invoices, cannot adjust cancelled invoices. Configured in `adjustment_limits_config`. |

# A — Target

## 1. What this process is about

After an invoice is issued, things can change. The customer was billed for service days they didn't get. A package price was applied wrongly. The customer disputed a line item and the operator agrees. A charge was missed and needs to be added retroactively. In all these cases, the operator issues a **credit note** (money back to the customer) or a **debit note** (additional money owed) to correct the original invoice. These corrective documents are legal records — they can't just be SQL updates.

ADJ-01 owns the admin workflow for proposing, approving, and applying these adjustments. The actual document generation (writing the credit or debit note row) is done by the `CreditNoteGenerator` and `DebitNoteGenerator` classes that live in BIL-02-GEN-01 Invoice Generation — ADJ-01 invokes them with the prepared context once approval clears. After the note exists, downstream consumers react: BIL-01 Charging Engine updates outstanding balance (POSTPAID) or wallet (PREPAID); DD_NOT-01 Notification Service notifies the customer.

#### What can be adjusted

| Direction | Note type | Effect on customer | Effect on operator |
|---|---|---|---|
| Customer is owed money / owes less | **Credit note** | POSTPAID: reduces outstanding balance on parent invoice (or applies as credit toward future invoices). PREPAID: credits the customer's wallet. | Revenue is decreased; financials reconcile the correction. |
| Customer owes more | **Debit note** | POSTPAID: increases outstanding balance (treated as an additional charge). PREPAID: debits the customer's wallet. | Revenue is increased; the missed charge is captured. |

#### Refunds outside the system (out of scope)

Refunding to the customer's original payment method (M-Pesa, bank, card) is NOT covered by this DD. ADJ-01's credit note refunds the customer's **local wallet only** (for PREPAID) or reduces outstanding (for POSTPAID). If an operator's process requires a refund to the original external payment method, that is a separate operations workflow handled outside the V3 system.

#### Scopes and when to use them

Every adjustment has a scope that determines how its amount is derived and how it ties back to the parent invoice. The three scopes serve distinct business purposes:

| Scope | What the admin selects | Amount comes from | Typical business use |
|---|---|---|---|
| **FULL** | The parent invoice itself | The invoice's total | Cancel/refund an entire invoice — customer wasn't served at all that month; whole invoice was wrong and needs to be redone |

| Scope | What the admin selects | Amount comes from | Typical business use |
|---|---|---|---|
| **LINE** | One or more specific line items of the parent invoice | Either the line's full total (line-full), or admin overrides downward for partial (line-partial) | **Line-full**: customer disputed one specific service (e.g., "I never use TV — credit the TV line"); a specific service had a wrong price applied. **Line-partial**: proration credit (e.g., "3 days outage on Internet out of 30 — credit 10% of the Internet line"). Anchored to a specific service for clean tax-and-service-category reporting. |
| **AMOUNT** | A free-form value with admin-specified service category | Admin types it | Goodwill credit not tied to any specific charge (retention gesture, settlement, general service-quality compensation). The admin must still pick a service category so the credit aggregates correctly in tax and service-type reports. |

**Why LINE supports partial amounts**: a common operator scenario is service disruption proration — the customer was charged for 30 days but only got 27 days of service, so credit 3/30 of that line. Pure full-line credit would force the operator to either credit the entire line (over-refund) or fall back to AMOUNT (losing the service-category link). LINE with optional per-line amount override handles this cleanly: the credit note's line still carries `service_category_code`, `package_ref`, `wallet_type_code` from the parent line, with `subtotal` scaled to the partial value and `tax_breakdown` scaled proportionally.

**Why AMOUNT still exists**: not every credit maps to a specific invoice line. A retention credit ("we're giving you 500 to stay") isn't really a refund of any charge — it's a gesture. Forcing the admin to attach it to an arbitrary line would misrepresent the credit's nature. AMOUNT lets the admin record what actually happened, with a service category for reporting cleanliness.

**Which to choose, summary**:

- "Refund the whole invoice" → FULL
- "Refund one specific service charge" → LINE (full per line)
- "Proration / partial refund of one service" → LINE (with per-line amount override)
- "Goodwill / settlement not tied to a specific charge" → AMOUNT

#### What ADJ-01 explicitly does not own

- **Generator classes** (`CreditNoteGenerator`, `DebitNoteGenerator`) — BIL-02-GEN-01 Invoice Generation
- **Tax recomputation on the adjustment amount** — PLM-CFG-02 Tax Configuration is called by the generator if needed
- **Outstanding balance update** (POSTPAID after credit note) — BIL-01 Charging Engine consumes `CreditNoteIssued` and updates
- **Wallet credit/debit** (PREPAID after credit/debit note) — BIL-05 Wallet and Topup consumes the event and credits/debits the wallet
- **Refund money movement to external payment methods** — out of scope; operators handle externally
- **Adjustments on tax invoices** — TAX-01 owns its own dual-approval signed-cancellation flow; ADJ-01 does NOT issue credit/debit notes against tax invoices
- **Notification of the customer** — DD_NOT-01 Notification Service consumes the event and notifies

## 2. Business rules and adjustment flow

Rules grouped by phase: `P` (proposal), `A` (approval), `G` (generation invocation), `L` (limits), `D` (downstream emission), `O` (admin operations).

#### Rule group P — Proposal

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-P-1** | An admin proposes an adjustment via REST endpoint or admin console UI. The proposal includes: parent invoice ID, adjustment direction (CREDIT or DEBIT), scope (FULL / LINE / AMOUNT), the proposed amount (or selected line items for LINE scope), reason code (from operator's catalog), free-text justification, and (for PREPAID) target wallet ID. | Admin proposal |
| **R-ADJ-01-P-2** | The system validates: parent invoice exists and is not CANCELLED; parent invoice is NOT a `TAX_INVOICE` (TAX-01 owns those); parent invoice's status allows adjustment (typically GENERATED, PENDING, PAID, OVERDUE); customer is not in a blocked state that prevents adjustments. | Validation |
| **R-ADJ-01-P-3** | **For FULL scope**: proposed amount equals the parent invoice's total. **For LINE scope**: admin selects one or more parent invoice lines; for each selected line, the admin can accept the line's full amount (line-full credit for disputes/wrong charges) or override to a smaller amount (line-partial credit for proration). The credit/debit note's line items inherit `service_category_code`, `package_ref`, `wallet_type_code` from the parent line; `tax_breakdown` is scaled proportionally if amount is overridden. **For AMOUNT scope**: admin enters a free-form total AND must select a `service_category_code` for the credit/debit note's single line — this preserves clean per-service tax reporting. Tax handling for AMOUNT is per operator config (some operators want the amount treated as tax-inclusive and decomposed; others treat it as a tax-exclusive base with tax computed on top). | Per-scope amount determination |
| **R-ADJ-01-P-4** | For PREPAID, the admin selects the target wallet (typically the one originally debited). If the parent invoice was for multiple wallets (hybrid grouping), admin must specify per-wallet allocation. The system validates the selected wallet exists and is associated with the same subscription as the parent invoice. | Target wallet selection |
| **R-ADJ-01-P-5** | The proposal is persisted in `adjustment_request` with status `PROPOSED` and the proposing admin's user ID, timestamp, and IP. An `AdjustmentProposed` event is emitted for audit. | Persistence |
| **R-ADJ-01-P-6** | An admin can cancel their own proposal before approval starts (status transitions PROPOSED → CANCELLED_BY_PROPOSER). | Cancel before approval |

#### Rule group A — Approval

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-A-1** | After proposal, ADJ-01 starts a Camunda process instance based on the operator's `adjustment_approval_workflow_ref`. The process variable carries the adjustment_request ID. The Camunda BPMN encodes the operator's approval steps. | Camunda kickoff |
| **R-ADJ-01-A-2** | The approval workflow is **per-operator configurable**. Operators choose from: zero-step (auto-approve, immediately move to APPROVED), single-step (one supervisor approves), multi-step (e.g., supervisor then finance manager then country head), threshold-based (auto-approve if amount ≤ threshold, single-step if ≤ higher threshold, multi-step otherwise). The workflow is implemented as Camunda BPMN files following the SUB-WF-FRAMEWORK-01 Subscription Workflow Framework pattern (BPMN deployment, task assignment, user-task UI integration). | Operator-configurable workflow |

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-A-3** | While approval is in progress, the `adjustment_request` status reflects the Camunda state: PENDING_APPROVAL (waiting on first/next approver), IN_APPROVAL (approver currently reviewing). The admin UI surfaces the current step and approver. | Visibility |
| **R-ADJ-01-A-4** | An approver can: APPROVE (proceeds to next step or final approval if last), REJECT (terminates the workflow; `adjustment_request` transitions to REJECTED with the rejection reason and reviewer ID), REQUEST_REVISION (returns to proposer for changes; proposer can re-submit or cancel). | Approver actions |
| **R-ADJ-01-A-5** | If all approval steps complete with APPROVE, the adjustment_request transitions to APPROVED. ADJ-01's Camunda service task invokes the appropriate GEN-01 generator (next rule group). | Final approval |
| **R-ADJ-01-A-6** | If the Camunda workflow has no human steps (zero-step config), the adjustment auto-approves on proposal — `adjustment_request` transitions PROPOSED → APPROVED immediately and generation is invoked synchronously. | Zero-step auto-approve |
| **R-ADJ-01-A-7** | An adjustment request awaiting approval can be cancelled by the proposer or by an admin with elevated privileges. Status transitions to CANCELLED with the cancelling user ID and reason. | Cancellation in approval |
| **R-ADJ-01-A-8** | Approval steps capture full audit: who approved/rejected, when, the comment text. Stored in `adjustment_approval_step`. | Audit |

#### Rule group G — Generation invocation

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-G-1** | When approval completes, ADJ-01 builds an `InvoiceGenerationContext` and calls `InvoiceGenerationService.generate(ctx)` in BIL-02 Core. The context carries the adjustment direction (CREDIT vs DEBIT), parent invoice reference, scope details, target wallet (PREPAID), reason code, approving user IDs, and the adjustment_request ID for back-reference. | Synchronous generation |
| **R-ADJ-01-G-2** | The generation dispatcher routes to `CreditNoteGenerator` or `DebitNoteGenerator` (both owned by BIL-02-GEN-01 Invoice Generation) based on direction. The generator returns the persisted credit or debit note row. | Generator dispatch |
| **R-ADJ-01-G-3** | On successful generation, ADJ-01 updates the `adjustment_request`: status transitions to APPLIED, the resulting credit_note_id or debit_note_id is stored, applied_at timestamp set. | Applied state |
| **R-ADJ-01-G-4** | On generation failure (DB error, transient downstream issue), the adjustment_request transitions to APPLICATION_FAILED with the error detail. Admin can retry manually from the admin console. The approval state is preserved — no need to re-approve. | Failure handling |
| **R-ADJ-01-G-5** | After generation succeeds, GEN-01's CreditNoteGenerator / DebitNoteGenerator emits `CreditNoteIssued` / `DebitNoteIssued` events with the full payload (the same way every generator does — this is BIL-02 Core's event publisher pattern). ADJ-01 does NOT emit a separate event; it consumes its own approval/application events for audit only. | Event emission via Core |

#### Rule group L — Limits

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-L-1** | Per-operator limits configured in `adjustment_limits_config`: maximum credit amount per single request, maximum total credit amount per customer per rolling period (e.g., 30 days), maximum debit amount per single request. Limits are validated at proposal time AND re-validated at approval completion (in case other adjustments were applied in the meantime). | Per-request and per-customer limits |
| **R-ADJ-01-L-2** | A credit note cannot exceed the parent invoice's remaining adjustable amount. Definition: parent_invoice.total minus (sum of prior credit notes applied) minus (sum of prior cancellations). The system computes this dynamically at proposal time and at re-validation. | Cap on credit |
| **R-ADJ-01-L-3** | For PREPAID debit notes: the system checks if the target wallet has sufficient balance. The operator's configured behavior decides: either `FAIL_IMMEDIATELY` (reject the adjustment with a clear error) or `MARK_PENDING_FOR_NEXT_TOPUP` (debit is recorded as PENDING and applied at next topup of that wallet). | Insufficient wallet balance for debit |
| **R-ADJ-01-L-4** | Cannot adjust a CANCELLED invoice. Cannot adjust a TAX_INVOICE (TAX-01 owns those). Cannot adjust an invoice that has an open in-progress adjustment request against it (one at a time, to avoid concurrent state confusion). | Hard prohibitions |
| **R-ADJ-01-L-5** | If a limit is exceeded at proposal time, the proposal is rejected with a clear error. If a limit is exceeded only at re-validation (after approval), the adjustment transitions to LIMIT_EXCEEDED_AT_APPLY; admin must review and decide whether to manually override (per operator policy) or cancel. | Limit handling |
| **R-ADJ-01-L-6** | Operators can configure exceptions: certain reason codes may be marked as "limit-exempt" via `reason_code.limit_exempt = true`. These bypass the standard limits but typically require a special approval workflow. | Limit-exempt reason codes |

#### Rule group D — Downstream emission

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-D-1** | Generation in GEN-01 emits `CreditNoteIssued` or `DebitNoteIssued` via BIL-02 Core's event publisher. The event payload includes the full credit/debit note (header + line items), parent invoice reference, target wallet ID (PREPAID), reason code, approving user IDs. | Note issued |
| **R-ADJ-01-D-2** | **BIL-01 Charging Engine** consumes `CreditNoteIssued` and `DebitNoteIssued`. For POSTPAID: updates outstanding balance on the parent invoice (credit reduces, debit increases). If the credit exceeds the parent's outstanding amount, the remainder is applied as a credit balance against the customer's next invoice. For PREPAID: BIL-01 forwards the credit/debit instruction to BIL-05 Wallet and Topup. | BIL-01 integration |
| **R-ADJ-01-D-3** | **BIL-05 Wallet and Topup** receives the wallet credit/debit instruction. Applies the credit (increasing wallet balance) or debit (decreasing wallet balance, or marking PENDING if insufficient per operator config). Emits its own wallet movement events. | BIL-05 integration (PREPAID) |
| **R-ADJ-01-D-4** | **DD_NOT-01 Notification Service** consumes `CreditNoteIssued` / `DebitNoteIssued` and notifies the customer per the operator's template (email, SMS, in-app), including a link to the PDF retrieval endpoint owned by BIL-02-READ-01 Invoice Read API. | Customer notification |

#### Rule group O — Admin operations

| ID | Rule | Triggered on |
|---|---|---|
| **R-ADJ-01-O-1** | Admin console provides per-operator views: pending proposals (PROPOSED or in approval), my approval queue (items waiting on me), applied adjustments (with filters by date range, reason code, amount range, customer), application failures (need attention). | Dashboards |
| **R-ADJ-01-O-2** | Admin can drill into any adjustment_request and see: full timeline (proposed, each approval step, applied, or rejected/cancelled), proposer and all approvers with comments, the resulting credit/debit note (if applied), the parent invoice reference. | Per-request detail |
| **R-ADJ-01-O-3** | Per-operator reporting endpoints provide aggregated metrics: total adjustment volume per reason code per period, top customers by credit volume, time-to-approval distribution, rejection rate, application failure rate. Used by finance reconciliation. | Reporting |
| **R-ADJ-01-O-4** | All admin actions (propose, approve, reject, request_revision, cancel, retry) are captured in `adjustment_request_history` and in BIL-02 Core's `invoice_history` (for the generated credit/debit note). | Audit trail |

## 3. Module owner and impacted modules

| Role | Module | What it does |
|---|---|---|
| **Owner** | New `billing-invoicing-adjustments` package within the `billing-invoicing` module on cluster C | Owns the proposal/approval workflow, the `adjustment_request` aggregate, the Camunda integration for approval, the limits enforcement, the admin REST endpoints. |
| **Generators (invoked, not owned)** | `CreditNoteGenerator`, `DebitNoteGenerator` in BIL-02-GEN-01 Invoice Generation | ADJ-01 builds the context and calls them via BIL-02 Core's `InvoiceGenerationService.generate(ctx)`. |
| **Camunda workflow engine** | Existing Camunda deployment used by SUB-WF-FRAMEWORK-01 Subscription Workflow Framework | ADJ-01 deploys operator-specific BPMN files for the approval workflow. |
| **Downstream consumers** | BIL-01 Charging Engine, BIL-05 Wallet and Topup, DD_NOT-01 Notification Service | React to `CreditNoteIssued` / `DebitNoteIssued` (emitted by GEN-01 via Core). |
| **Frontend** | Admin Console — Adjustments dashboard, proposal form, approval queue, per-request detail, reporting | Built on BIL-02 Core's admin shell. |
| **Engines** | Camunda for approval workflow. No Drools in ADJ-01 (limits are simple config rules in tables, not rule-engine rules). | — |

## 4. Foundation references

DB → `FOUNDATION_DB.md` · Kafka → `FOUNDATION_KAFKA.md` · Camunda → `FOUNDATION_CAMUNDA.md` · Auth → `FOUNDATION_AUTH.md` (introduces `ADJUSTMENT_PROPOSER` and approval roles). Framework: BIL-02 Invoicing (Core) + SUB-WF-FRAMEWORK-01 Subscription Workflow Framework (for the Camunda integration pattern).

# Technical design

## Module placement

Co-located in `billing-invoicing` module on cluster C. New sub-package `adjustments`. Adds 4 new dedicated tables (`adjustment_request`, `adjustment_approval_step`, `adjustment_limits_config`, `adjustment_request_history`) and uses Core's existing tables and the existing Camunda deployment.

## Code structure

```
modules/billing-invoicing/
  src/main/java/com/sophix/billing/invoicing/adjustments/
    service/
      AdjustmentRequestService.java     ← proposal, validation, lifecycle transitions
```

```
    AdjustmentApprovalService.java    ← Camunda integration, approver actions
    AdjustmentApplicationService.java ← invokes GEN-01 generators after approval
    AdjustmentLimitsService.java      ← per-operator limit enforcement
  camunda/
    AdjustmentApprovalDelegate.java   ← service tasks for Camunda processes
    AdjustmentApprovalTaskHandler.java
  rest/
    AdjustmentAdminResource.java      ← admin REST endpoints
    AdjustmentApprovalResource.java   ← approver REST endpoints
  domain/
    AdjustmentRequest.java
    AdjustmentApprovalStep.java
    AdjustmentScope.java              ← enum: FULL, LINE, AMOUNT
    AdjustmentDirection.java          ← enum: CREDIT, DEBIT
    AdjustmentRequestStatus.java      ← enum: PROPOSED, PENDING_APPROVAL, ...
  repository/
    AdjustmentRequestRepository.java
    AdjustmentApprovalStepRepository.java
    AdjustmentLimitsConfigRepository.java
src/main/resources/
  config/liquibase/changelog/billing-invoicing/
    060_adjustment_request.xml
    061_adjustment_approval_step.xml
    062_adjustment_limits_config.xml
    063_adjustment_request_history.xml
  bpmn/
    adjustment_approval_template.bpmn    ← reference template; operators deploy their own
```

## Adjustment end-to-end flow

The canonical happy path from admin proposal to applied credit/debit note:

```
Step 1 — Admin proposes
  Admin user logs into the admin console, opens the parent invoice, clicks "Propose adjustment."
  Form fields: direction (CREDIT/DEBIT), scope (FULL/LINE/AMOUNT), amount or selected lines,
  reason code, free-text justification, (PREPAID only) target wallet.
  POSTs to /api/adjustments — AdjustmentRequestService.propose(...)

Step 2 — Validation
  AdjustmentRequestService:
    - Load parent invoice; reject if CANCELLED, TAX_INVOICE, or has an open adjustment
    - Compute parent's remaining adjustable amount; reject if proposed exceeds it (R-ADJ-01-L-2)
    - Check per-operator limits via AdjustmentLimitsService (R-ADJ-01-L-1)
    - For PREPAID: validate target wallet belongs to same subscription as parent invoice
    - For PREPAID debit: check wallet balance per operator config (R-ADJ-01-L-3)
    - Persist adjustment_request row with status PROPOSED
    - Emit internal AdjustmentProposed event for audit

Step 3 — Approval workflow kickoff
  AdjustmentApprovalService.startApproval(adjustment_request_id):
    - Look up operator's adjustment_approval_workflow_ref (e.g., a Camunda processDefinitionKey)
    - Start Camunda process instance with adjustment_request_id as process variable
    - Camunda handles the rest:
        - If zero-step: process completes immediately, advances to Step 4 via service task
        - If single-step: creates user task for first approver; waits
        - If multi-step / threshold-based: orchestrates as BPMN defines

  While Camunda runs:
    - Approvers see their pending tasks in the admin console approval queue
    - Approver actions: APPROVE, REJECT, REQUEST_REVISION (REST endpoints on AdjustmentApprovalResource)
    - Each step writes adjustment_approval_step row + updates adjustment_request status

  Approver clicks APPROVE on the final step (or no human steps required):
    - Camunda BPMN's final service task invokes AdjustmentApprovalDelegate.onApprovalComplete
    - adjustment_request transitions to APPROVED

Step 4 — Application (generation invocation)
  AdjustmentApplicationService.apply(adjustment_request_id):
    - Re-validate limits (R-ADJ-01-L-1) — state may have changed during approval
    - If re-validation fails → adjustment_request → LIMIT_EXCEEDED_AT_APPLY; admin notified
    - Build InvoiceGenerationContext:
        invoice_type_code = CREDIT_NOTE or DEBIT_NOTE
        triggering_event_ref = adjustment_request_id
        parent_invoice_id, scope, target_wallet (PREPAID), reason_code, approving_user_ids
    - Call BIL-02 Core's InvoiceGenerationService.generate(ctx)
        → Dispatches to CreditNoteGenerator or DebitNoteGenerator (GEN-01)
        → Generator builds + writes the note row, emits CreditNoteIssued/DebitNoteIssued event
    - Update adjustment_request: status = APPLIED, credit_note_id/debit_note_id set, applied_at = now
    - Internal AdjustmentApplied event emitted for audit

Step 5 — Downstream consumption
  CreditNoteIssued / DebitNoteIssued → consumed by:
```

```
    - BIL-01 Charging Engine:
        POSTPAID: updates outstanding balance on parent invoice (and applies excess as future credit)
        PREPAID: forwards instruction to BIL-05
    - BIL-05 Wallet and Topup (PREPAID):
        Applies wallet credit (increase balance) or debit (decrease, or mark PENDING if insufficient)
        Emits wallet movement events for ledger
    - DD_NOT-01 Notification Service:
        Renders the credit/debit note PDF; sends to customer per operator's template
    - analytics: aggregates for reporting

Step 6 — Customer sees the result
  POSTPAID: next time customer views their account, parent invoice's outstanding balance reflects
            the credit (reduced) or debit (increased). If credit exceeded what was owed, the
            difference appears as available balance against future invoices.
  PREPAID: wallet balance reflects the change immediately. If debit was deferred, customer sees
           a pending debit that'll apply at next topup.
  Both: customer receives notification with PDF.
```

**Failure paths**:

| Where it fails | Lands in | Recovery |
|---|---|---|
| Validation at proposal | Proposal rejected with clear error to admin | Admin corrects and re-proposes |
| Limit exceeded at proposal | Proposal rejected | Admin reduces amount or escalates (if reason code is limit-exempt) |
| Camunda workflow stuck (e.g., approver unresponsive) | Admin console flags long-pending requests; admin can re-assign approver or cancel | Operations review |
| Limit re-validation fails at apply | adjustment_request → LIMIT_EXCEEDED_AT_APPLY | Admin reviews, may override or cancel |
| Generation failure (DB error, downstream issue) | adjustment_request → APPLICATION_FAILED | Admin clicks retry; preserves approval state |
| Generation succeeds but BIL-01/BIL-05 fails to apply | Note exists but balance not updated | BIL-01/BIL-05 own their own retry; eventual consistency |

## Approval workflow samples (Camunda BPMN)

Each operator deploys their own BPMN process for the adjustment approval workflow. The BPMN is referenced by `adjustment_approval_workflow_ref` per operator. The framework supports any BPMN expressible in Camunda; the four samples below cover the typical patterns operators choose between. Each sample shows the BPMN as a flow plus the resulting `adjustment_request` status transitions.

#### Sample 1 — Zero-step (auto-approve everything)

Used by operators that trust their proposers (e.g., a senior billing admin role with no separate approver tier) or for very low-stakes cases. The proposal is auto-approved on submission.

```
[Start: AdjustmentProposed]
    ■
    ▼
[Service Task: AdjustmentApprovalDelegate.onApprovalComplete]
    ■  (invokes AdjustmentApplicationService.apply)
    ▼
[End]
```

Status transitions: `PROPOSED` → `APPROVED` → `APPLIED` (all within milliseconds of proposal).

No `adjustment_approval_step` rows are written (no human action). The audit trail relies on the proposer record and the resulting credit/debit note.

#### Sample 2 — Single-step (one supervisor)

Used by smaller operators or for moderate amounts. Proposer submits; one supervisor reviews and approves or rejects.

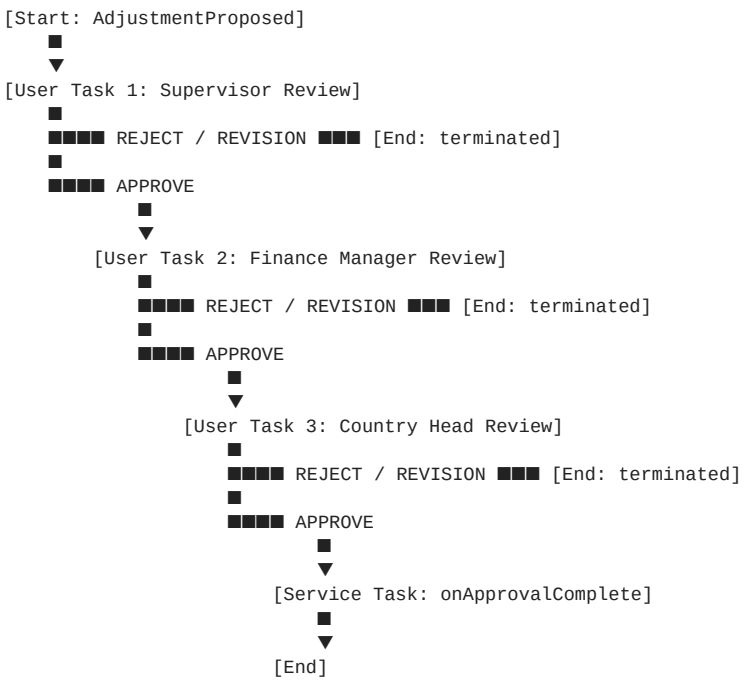
```
[Start: AdjustmentProposed]
    ■
    ▼
[User Task: Supervisor Review]
    ■  (assigned to users with SUPERVISOR role for this operator)
    ■
    ■■■■ action = REJECT ■■■■■■■■ [End: Rejected]
    ■
    ■■■■ action = REQUEST_REVISION ■■ [End: Returned to proposer]
    ■
    ■■■■ action = APPROVE
            ■
            ▼
        [Service Task: onApprovalComplete]
            ■
```


Status transitions on the happy path: `PROPOSED` → `PENDING_APPROVAL` (waiting for supervisor) → `IN_APPROVAL` (supervisor opens the task) → `APPROVED` (supervisor clicks APPROVE) → `APPLIED`.

One `adjustment_approval_step` row per supervisor action.

#### Sample 3 — Multi-step (supervisor → finance manager → country head)

Used by operators that want layered approval for accountability. Each step must approve before the next sees the request. Any step can REJECT or REQUEST_REVISION to terminate or send back.

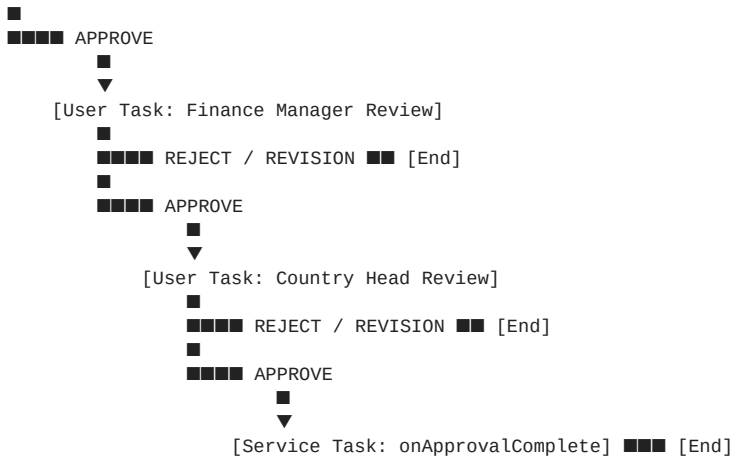



Status transitions: `PROPOSED` → `PENDING_APPROVAL` (step 1) → `IN_APPROVAL` → `PENDING_APPROVAL` (step 2) → `IN_APPROVAL` → `PENDING_APPROVAL` (step 3) → `IN_APPROVAL` → `APPROVED` → `APPLIED`.

Three `adjustment_approval_step` rows (step_number 1, 2, 3) — one per approval action.

#### Sample 4 — Threshold-based (the most common real-world choice)

Combines the previous patterns based on the proposed amount. Small amounts auto-approve; medium amounts need one supervisor; large amounts go through the full chain.




The gateway conditions read from `adjustment_limits_config.auto_approve_threshold` and `adjustment_limits_config.single_step_threshold` via Camunda process variables set at workflow kickoff.

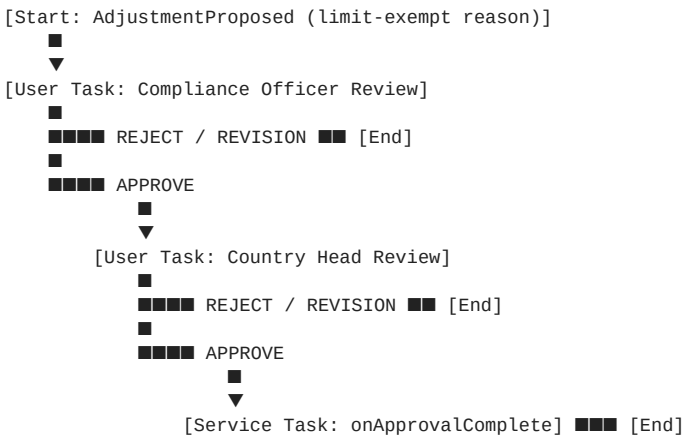
Illustrative configuration (per operator + direction + currency):

- Auto-approve: amounts up to 500 (e.g., small goodwill credits) — no human review
- Single-step: amounts 501 to 5,000 — one supervisor
- Multi-step: amounts above 5,000 — supervisor + finance manager + country head

Status transitions depend on which branch the amount takes.

#### Sample 5 — Limit-exempt special workflow

Used when a reason code is flagged `limit_exempt = true` (e.g., `COURT_ORDER`, `REGULATORY_REFUND`). The standard limits are bypassed, but the approval is typically stricter — often requiring legal/compliance sign-off in addition to finance.




Selected by the proposer if the reason code is limit-exempt — the system routes to this BPMN instead of the standard threshold-based one. Per-operator configuration via `reason_code.approval_workflow_override_ref`.

#### What the actual artifacts look like

The samples above are flow diagrams. Below are the concrete artifacts a developer or operator engineer produces to deploy them. These are the things you would actually write and commit to source control.

##### 1. The operator configuration row

The operator's choice of approval workflow is a single row in `bil02_invoice_type_operator_config` (or a dedicated table — names vary) that points to a Camunda process definition by key. Illustrative:

```
INSERT INTO adjustment_workflow_operator_config (
    id, operator_code, direction, currency,
    approval_workflow_process_key,
    auto_approve_threshold, single_step_threshold,
    max_per_request, max_per_customer_per_period, period_days
) VALUES (
    'awoc_01HZ...', 'WIK', 'CREDIT', 'KES',
    'adjustment_approval_threshold_v1',   -- Camunda processDefinitionKey
    500.00,                                -- auto-approve up to this
    5000.00,                               -- single-step up to this
    50000.00,                              -- absolute cap per request
```

```
    100000.00,                          -- cap per customer per 30 days
    30
);
```

When a proposal is submitted, `AdjustmentApprovalService` looks up this row by `(operator_code, direction, currency)`, reads the `approval_workflow_process_key`, and starts a Camunda process by that key with the row's thresholds as process variables.

##### 2. The BPMN file (Sample 4 — threshold-based, the most common)

The BPMN is XML that Camunda interprets. Operators commit one BPMN file per process key they reference. A trimmed-but-real example showing the threshold-based pattern:

```
<?xml version="1.0" encoding="UTF-8"?>
<bpmn:definitions xmlns:bpmn="http://www.omg.org/spec/BPMN/20100524/MODEL"
                  xmlns:camunda="http://camunda.org/schema/1.0/bpmn"
                  targetNamespace="http://sophix.com/bpmn/adjustment">

  <bpmn:process id="adjustment_approval_threshold_v1" name="Adjustment Approval (threshold)"
                isExecutable="true" camunda:historyTimeToLive="P3650D">

    <bpmn:startEvent id="StartEvent_Proposed" name="Adjustment Proposed"/>

    <bpmn:sequenceFlow sourceRef="StartEvent_Proposed" targetRef="Gateway_AutoApprove"/>

    <bpmn:exclusiveGateway id="Gateway_AutoApprove" name="Amount ≤ auto-approve threshold?"/>

    <bpmn:sequenceFlow sourceRef="Gateway_AutoApprove" targetRef="ServiceTask_AutoApprove">
      <bpmn:conditionExpression xsi:type="bpmn:tFormalExpression">
        ${proposedAmount &lt;= autoApproveThreshold}
      </bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <bpmn:sequenceFlow sourceRef="Gateway_AutoApprove" targetRef="Gateway_SingleStep">
      <bpmn:conditionExpression xsi:type="bpmn:tFormalExpression">
        ${proposedAmount &gt; autoApproveThreshold}
      </bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <bpmn:exclusiveGateway id="Gateway_SingleStep" name="Amount ≤ single-step threshold?"/>

    <bpmn:sequenceFlow sourceRef="Gateway_SingleStep" targetRef="UserTask_SupervisorOnly">
      <bpmn:conditionExpression xsi:type="bpmn:tFormalExpression">
        ${proposedAmount &lt;= singleStepThreshold}
      </bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <bpmn:sequenceFlow sourceRef="Gateway_SingleStep" targetRef="UserTask_SupervisorMulti">
      <bpmn:conditionExpression xsi:type="bpmn:tFormalExpression">
        ${proposedAmount &gt; singleStepThreshold}
      </bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <!-- Single-step branch -->
    <bpmn:userTask id="UserTask_SupervisorOnly" name="Supervisor Review"
                   camunda:candidateGroups="${operatorCode}_SUPERVISOR"/>
    <bpmn:sequenceFlow sourceRef="UserTask_SupervisorOnly" targetRef="Gateway_DecisionSingle"/>

    <bpmn:exclusiveGateway id="Gateway_DecisionSingle"/>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionSingle" targetRef="ServiceTask_AutoApprove">
      <bpmn:conditionExpression>${approverAction == 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionSingle" targetRef="EndEvent_Rejected">
      <bpmn:conditionExpression>${approverAction != 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <!-- Multi-step branch -->
    <bpmn:userTask id="UserTask_SupervisorMulti" name="Supervisor Review"
                   camunda:candidateGroups="${operatorCode}_SUPERVISOR"/>
    <bpmn:sequenceFlow sourceRef="UserTask_SupervisorMulti" targetRef="Gateway_DecisionStep1"/>

    <bpmn:exclusiveGateway id="Gateway_DecisionStep1"/>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionStep1" targetRef="UserTask_FinanceManager">
      <bpmn:conditionExpression>${approverAction == 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionStep1" targetRef="EndEvent_Rejected">
      <bpmn:conditionExpression>${approverAction != 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <bpmn:userTask id="UserTask_FinanceManager" name="Finance Manager Review"
                   camunda:candidateGroups="${operatorCode}_FINANCE_MANAGER"/>
    <bpmn:sequenceFlow sourceRef="UserTask_FinanceManager" targetRef="Gateway_DecisionStep2"/>
```

```
    <bpmn:exclusiveGateway id="Gateway_DecisionStep2"/>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionStep2" targetRef="UserTask_CountryHead">
      <bpmn:conditionExpression>${approverAction == 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionStep2" targetRef="EndEvent_Rejected">
      <bpmn:conditionExpression>${approverAction != 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <bpmn:userTask id="UserTask_CountryHead" name="Country Head Review"
                   camunda:candidateGroups="${operatorCode}_COUNTRY_HEAD"/>
    <bpmn:sequenceFlow sourceRef="UserTask_CountryHead" targetRef="Gateway_DecisionFinal"/>

    <bpmn:exclusiveGateway id="Gateway_DecisionFinal"/>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionFinal" targetRef="ServiceTask_AutoApprove">
      <bpmn:conditionExpression>${approverAction == 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>
    <bpmn:sequenceFlow sourceRef="Gateway_DecisionFinal" targetRef="EndEvent_Rejected">
      <bpmn:conditionExpression>${approverAction != 'APPROVE'}</bpmn:conditionExpression>
    </bpmn:sequenceFlow>

    <!-- Final application service task -->
    <bpmn:serviceTask id="ServiceTask_AutoApprove" name="Apply Adjustment"
                      camunda:delegateExpression="${adjustmentApprovalDelegate}"/>
    <bpmn:sequenceFlow sourceRef="ServiceTask_AutoApprove" targetRef="EndEvent_Applied"/>

    <bpmn:endEvent id="EndEvent_Applied" name="Applied"/>
    <bpmn:endEvent id="EndEvent_Rejected" name="Rejected"/>

  </bpmn:process>
</bpmn:definitions>
```

What this XML expresses:

- A process with id `adjustment_approval_threshold_v1` — this is what `approval_workflow_process_key` in the config row references
- Process variables expected at start: `proposedAmount`, `autoApproveThreshold`, `singleStepThreshold`, `operatorCode` — set by `AdjustmentApprovalService` when starting the instance
- User tasks assigned to candidate groups computed dynamically from `operatorCode` (e.g., `WIK_SUPERVISOR`) — operator-specific role groups managed via FOUNDATION_AUTH.md
- Gateways branch on the `approverAction` variable that each user-task completion sets (APPROVE / REJECT / REQUEST_REVISION)
- A single final service task calls a Spring bean named `adjustmentApprovalDelegate` (the Java class shown below) to actually invoke generation

For Sample 1 (zero-step), the BPMN is dramatically simpler — start event, service task, end event, no gateways:

```
<bpmn:process id="adjustment_approval_auto_v1" isExecutable="true">
  <bpmn:startEvent id="Start"/>
  <bpmn:sequenceFlow sourceRef="Start" targetRef="Apply"/>
  <bpmn:serviceTask id="Apply" name="Apply Adjustment"
                    camunda:delegateExpression="${adjustmentApprovalDelegate}"/>
  <bpmn:sequenceFlow sourceRef="Apply" targetRef="End"/>
  <bpmn:endEvent id="End"/>
</bpmn:process>
```

For Samples 2, 3, and 5, the BPMN is a subset of Sample 4's structure — fewer gateways, fewer user tasks. Same building blocks.

##### 3. The Java service that kicks off the workflow

`AdjustmentApprovalService` is the Spring bean ADJ-01 owns. It reads the operator's config, sets process variables, and starts the Camunda process. Skeleton:

```
@Service
public class AdjustmentApprovalService {

    private final RuntimeService runtimeService;  // Camunda API
    private final AdjustmentRequestRepository requestRepository;
    private final AdjustmentWorkflowConfigRepository configRepository;

    @Transactional
    public void startApproval(String adjustmentRequestId) {
        AdjustmentRequest request = requestRepository.findById(adjustmentRequestId)
            .orElseThrow();

        AdjustmentWorkflowConfig config = configRepository.findByKey(
            request.getOperatorCode(),
```

```
                request.getDirection(),
                request.getCurrency()
            ).orElseThrow();

            // Process variables — match what BPMN expressions reference
            Map<String, Object> variables = Map.of(
                "adjustmentRequestId", adjustmentRequestId,
                "operatorCode",        request.getOperatorCode(),
                "proposedAmount",      request.getProposedAmount(),
                "reasonCode",          request.getReasonCode(),
                "autoApproveThreshold", config.getAutoApproveThreshold(),
                "singleStepThreshold",  config.getSingleStepThreshold()
            );

            ProcessInstance instance = runtimeService.startProcessInstanceByKey(
                config.getApprovalWorkflowProcessKey(),  // e.g., "adjustment_approval_threshold_v1"
                adjustmentRequestId,                      // business key
                variables
            );

            // Persist Camunda instance ID back on the request for cross-reference
            request.setCamundaProcessInstanceId(instance.getId());
            request.setStatus(AdjustmentRequestStatus.PENDING_APPROVAL);
            requestRepository.save(request);
        }
    }
```

##### 4. The Java delegate that completes the workflow

When the BPMN reaches its final `serviceTask`, Camunda invokes the `adjustmentApprovalDelegate` Spring bean. This delegate triggers generation:

```
    @Component("adjustmentApprovalDelegate")
    public class AdjustmentApprovalDelegate implements JavaDelegate {

        private final AdjustmentApplicationService applicationService;

        @Override
        public void execute(DelegateExecution execution) throws Exception {
            String adjustmentRequestId = (String) execution.getVariable("adjustmentRequestId");
            applicationService.apply(adjustmentRequestId);
            // applicationService.apply():
            //   1. Re-validates limits (per R-ADJ-01-L-1)
            //   2. Builds InvoiceGenerationContext
            //   3. Calls InvoiceGenerationService.generate(ctx) -> dispatches to CreditNoteGenerator
            //   4. Updates adjustment_request: APPROVED -> APPLIED
        }
    }
```

##### 5. The REST endpoint that user tasks call

When an approver clicks APPROVE / REJECT / REQUEST_REVISION in the admin console, the UI calls a REST endpoint. The endpoint resolves the current Camunda user task for this adjustment and completes it with the action as a process variable:

```
    @RestController
    @RequestMapping("/api/adjustments")
    public class AdjustmentApprovalResource {

        private final TaskService taskService;  // Camunda API
        private final AdjustmentApprovalStepRepository stepRepository;

        @PostMapping("/{id}/approve")
        @PreAuthorize("hasRole('ADJUSTMENT_APPROVER')")
        public void approve(@PathVariable String id, @RequestBody ApprovalActionRequest req) {
            completeTask(id, "APPROVE", req.getComment());
        }

        @PostMapping("/{id}/reject")
        @PreAuthorize("hasRole('ADJUSTMENT_APPROVER')")
        public void reject(@PathVariable String id, @RequestBody ApprovalActionRequest req) {
            completeTask(id, "REJECT", req.getComment());
        }

        @PostMapping("/{id}/request-revision")
        @PreAuthorize("hasRole('ADJUSTMENT_APPROVER')")
        public void requestRevision(@PathVariable String id, @RequestBody ApprovalActionRequest req) {
            completeTask(id, "REQUEST_REVISION", req.getComment());
        }

        private void completeTask(String adjustmentRequestId, String action, String comment) {
            // Find the active user task for this adjustment's process instance
```

```
                Task task = taskService.createTaskQuery()
                    .processInstanceBusinessKey(adjustmentRequestId)
                    .taskAssignee(currentUserId())
                    .singleResult();
                if (task == null) throw new NotFoundException("No active task for this user");

                // Record the action in our own audit table BEFORE completing the Camunda task
                AdjustmentApprovalStep step = new AdjustmentApprovalStep(
                    adjustmentRequestId, currentUserId(), action, comment, Instant.now()
                );
                stepRepository.save(step);

                // Complete the Camunda task with action as a variable — BPMN gateways branch on this
                taskService.complete(task.getId(), Map.of("approverAction", action));
            }
        }
```

##### 6. Deployment

BPMN files live in `src/main/resources/bpmn/` and are auto-deployed to Camunda at application startup (standard Spring Boot Camunda integration). Operators commit their own BPMN files into operator-specific overlays; the build system packages the right BPMN per operator deployment. The operator's `adjustment_workflow_operator_config` row points to the deployed process by key.

#### Common building blocks across all samples

All approval BPMNs share the same patterns:

- **Start event**: `AdjustmentProposed` with the adjustment_request_id as a process variable
- **User tasks**: assigned by Camunda role (SUPERVISOR, FINANCE_MANAGER, COUNTRY_HEAD, COMPLIANCE_OFFICER) — operator configures which users hold which role via FOUNDATION_AUTH.md
- **Each user task** exposes three actions: APPROVE / REJECT / REQUEST_REVISION (the same admin REST endpoints regardless of which task)
- **Final service task**: always `AdjustmentApprovalDelegate.onApprovalComplete`, which invokes `AdjustmentApplicationService.apply` to trigger generation
- **Rejection or revision terminates the process**: process ends, status reflects outcome (REJECTED / CANCELLED_BY_PROPOSER if proposer cancels after revision request)
- **Audit**: Camunda's internal history captures the BPMN-level events; ADJ-01's `adjustment_approval_step` table captures the business-level approval actions

#### How operators choose between patterns

Operators pick based on their organizational structure and the financial volume of their adjustments. Typical choices:

- **Small/agile operators**: zero-step or single-step — they trust their admin team
- **Mid-size operators**: threshold-based — auto-approve small, escalate large
- **Large/enterprise operators**: multi-step or threshold-based with high tiers — financial controls require multiple sign-offs for sizeable adjustments
- **Operators in regulated markets**: limit-exempt special workflow for compliance-sensitive cases (court orders, regulator-mandated refunds)

The DD describes the capability; the choice is the operator's. The framework supports moving between patterns by deploying a different BPMN — no code changes in ADJ-01.

## Data model

#### `adjustment_request`

The aggregate root for the adjustment workflow lifecycle.

| Column | Type | Notes |
|---|---|---|
| `id` | TEXT PK | ULID |
| `operator_code` | TEXT NOT NULL | |
| `parent_invoice_id` | TEXT NOT NULL | FK to `invoice` |
| `customer_id` | TEXT NOT NULL | Denormalized from parent invoice for fast lookup |
| `subscription_id` | TEXT NOT NULL | Denormalized |

| Column | Type | Notes |
|---|---|---|
| direction | TEXT NOT NULL | CREDIT / DEBIT |
| scope | TEXT NOT NULL | FULL / LINE / AMOUNT |
| proposed_amount | NUMERIC(18,4) NOT NULL | The total (incl. tax) proposed |
| proposed_line_items | JSONB NULL | For LINE scope: array of objects [{parent_line_item_id, amount_override?}]. If amount_override is omitted, the line's full total is used; if present, it's the partial value to credit/debit from that line (must be ≤ line total). |
| amount_service_category_code | TEXT NULL | For AMOUNT scope only: the service category to tag the credit/debit note's line item with (preserves per-service reporting). NULL for FULL and LINE scopes (categories come from parent lines). |
| target_wallet_id | TEXT NULL | For PREPAID: which wallet the credit/debit applies to |
| reason_code | TEXT NOT NULL | From operator's reason code catalog |
| justification | TEXT NOT NULL | Free-text from proposer |
| status | TEXT NOT NULL | PROPOSED / PENDING_APPROVAL / IN_APPROVAL / APPROVED / APPLIED / REJECTED / CANCELLED_BY_PROPOSER / CANCELLED / APPLICATION_FAILED / LIMIT_EXCEEDED_AT_APPLY |
| proposed_by | TEXT NOT NULL | Admin user ID |
| proposed_at | TIMESTAMPTZ NOT NULL | |
| camunda_process_instance_id | TEXT NULL | The Camunda instance handling approval |
| approved_at | TIMESTAMPTZ NULL | When final approval completed |
| applied_at | TIMESTAMPTZ NULL | When the note was generated |
| credit_note_id | TEXT NULL | FK to invoice (if direction=CREDIT and applied) |
| debit_note_id | TEXT NULL | FK to invoice (if direction=DEBIT and applied) |
| rejection_reason | TEXT NULL | If status=REJECTED |
| application_error | TEXT NULL | If status=APPLICATION_FAILED |
| metadata | JSONB NULL | Operator-specific extensions |

Indexes: (operator_code, status), (parent_invoice_id), (customer_id, proposed_at DESC), (camunda_process_instance_id). P10Y retention.

**Sample data:**

| id | operator_code | parent_invoice_id | direction | scope | proposed_amount | target_wallet_id | reason_code | status |
|---|---|---|---|---|---|---|---|---|
| adj_01HZQX1A... | WIK | inv_01HZQ... | CREDIT | LINE | 2,000.00 | NULL (POSTPAID) | SERVICE_DISPUTE | APPLIED |
| adj_01HZQX1B... | WIK | inv_01HZR... | CREDIT | LINE | 400.00 | wal_01HZ... | SERVICE_DISRUPTION | PENDING_APPROVAL |
| adj_01HZQX1C... | WIK | inv_01HZS... | CREDIT | AMOUNT | 500.00 | wal_01HZ... | GOODWILL_RETENTION | APPROVED |
| adj_01HZQX1D... | WIK | inv_01HZT... | DEBIT | AMOUNT | 350.00 | NULL (POSTPAID) | MISSED_CHARGE | APPLIED |

How to read: first is a POSTPAID full-line credit — customer disputed the TV service, admin selected the TV line (worth 2,000), no amount override (line-full). Second is a PREPAID line-partial — admin selected the Internet line and overrode the amount to 400 for a 3-day proration of a 30-day cycle. Third is a PREPAID free-form goodwill credit for retention, tagged with an AMOUNT-scope service category for reporting. Fourth is a POSTPAID missed-charge debit applied via AMOUNT scope (no original line to anchor to since the charge was missed entirely).

#### adjustment_approval_step

One row per approval action (approve, reject, request_revision).

| Column | Type | Notes |
|---|---|---|
| id | TEXT PK | ULID |

| Column | Type | Notes |
|---|---|---|
| adjustment_request_id | TEXT NOT NULL | FK |
| step_number | INTEGER NOT NULL | 1, 2, 3 — sequential |
| step_role | TEXT NOT NULL | Configured per BPMN (e.g., SUPERVISOR, FINANCE_MANAGER, COUNTRY_HEAD) |
| actor_user_id | TEXT NOT NULL | Approver |
| action | TEXT NOT NULL | APPROVE / REJECT / REQUEST_REVISION |
| comment | TEXT NULL | Approver's note |
| acted_at | TIMESTAMPTZ NOT NULL | |

Indexes: (adjustment_request_id, step_number) UNIQUE; (actor_user_id, acted_at DESC). P10Y retention.

#### adjustment_limits_config

Per-operator configurable limits.

| Column | Type | Notes |
|---|---|---|
| id | TEXT PK | ULID |
| operator_code | TEXT NOT NULL | |
| direction | TEXT NOT NULL | CREDIT / DEBIT |
| max_per_request | NUMERIC(18,4) NULL | Absolute cap per single adjustment |
| max_per_customer_per_period | NUMERIC(18,4) NULL | Cap per customer in rolling period |
| period_days | INTEGER NULL | Rolling period (typically 30) |
| currency | TEXT NOT NULL | Limits are currency-specific |
| auto_approve_threshold | NUMERIC(18,4) NULL | Zero-step auto-approve below this amount (referenced by BPMN if threshold-based) |
| single_step_threshold | NUMERIC(18,4) NULL | Single-step below this amount; multi-step above |

Indexes: (operator_code, direction, currency) UNIQUE.

#### adjustment_request_history

Change history (audit shadow). Same pattern as other history tables in Core.

| Column | Type | Notes |
|---|---|---|
| id | TEXT PK | ULID |
| adjustment_request_id | TEXT NOT NULL | FK |
| changed_at | TIMESTAMPTZ NOT NULL | |
| changed_by | TEXT NOT NULL | User ID or system |
| field | TEXT NOT NULL | E.g., status, proposed_amount |
| old_value | TEXT NULL | |
| new_value | TEXT NULL | |
| reason | TEXT NULL | If applicable |

Indexes: (adjustment_request_id, changed_at DESC). P10Y retention.

## API surface — what ADJ-01 exposes

| Mechanism | Used for | Examples |
|---|---|---|
| **REST (admin-facing — proposers)** | Propose, cancel, retry adjustments | POST /api/adjustments, POST /api/adjustments/{id}/cancel |
| **REST (admin-facing — approvers)** | Approve, reject, request revision | POST /api/adjustments/{id}/approve, POST /api/adjustments/{id}/reject |
| **REST (admin-facing — ops/reporting)** | List, drill into request, reporting | GET /api/adjustments, GET /api/adjustments/{id} |
| **Camunda BPMN** | Approval workflow per operator | Camunda process instances; admin sees user tasks in Camunda Tasklist (or integrated admin console UI) |

| Mechanism | Used for | Examples |
|---|---|---|
| **In-process Java call** | Generation invocation after approval | `InvoiceGenerationService.generate(ctx)` to GEN-01 |
| **Kafka emission via Core** | Generated note events emitted by GEN-01 (NOT by ADJ-01 directly) | `CreditNoteIssued`, `DebitNoteIssued` |

**What ADJ-01 does NOT expose as REST**:

| Capability | Where it lives instead |
|---|---|
| `GET /api/credit-notes/{id}` (read credit note) | **BIL-02-READ-01 Invoice Read API** (unified invoice read API) |
| `GET /api/credit-notes/{id}/pdf` | **BIL-02-READ-01 Invoice Read API** |
| Direct invocation of CreditNoteGenerator | **BIL-02-GEN-01 Invoice Generation** owns the generators; ADJ-01 invokes via Core's service |
| Wallet credit/debit endpoints | **BIL-05 Wallet and Topup** |
| Outstanding balance update | **BIL-01 Charging Engine** |
| Adjustment of tax invoices | **BIL-02-TAX-01 Tax Invoice & Gateway** (dual-approval signed cancellation; not credit/debit notes) |

#### REST endpoints (admin-facing)

```
POST   /api/adjustments                          ← propose
GET    /api/adjustments                          ← list (filterable by operator, status, customer, date)
GET    /api/adjustments/{id}                     ← per-request detail with full timeline
POST   /api/adjustments/{id}/cancel              ← proposer cancels before approval starts
POST   /api/adjustments/{id}/approve             ← approver approves (current step)
POST   /api/adjustments/{id}/reject              ← approver rejects (terminates workflow)
POST   /api/adjustments/{id}/request-revision    ← approver returns to proposer
POST   /api/adjustments/{id}/retry-application   ← admin retries after APPLICATION_FAILED
POST   /api/adjustments/{id}/override-limit      ← admin overrides LIMIT_EXCEEDED_AT_APPLY (requires special privilege)

GET    /api/adjustments/reporting/by-reason      ← aggregated by reason_code
GET    /api/adjustments/reporting/by-customer    ← top customers by credit volume
GET    /api/adjustments/reporting/time-to-apply  ← workflow latency metrics
```

## Cross-DD coordination

| Touched | What ADJ-01 owns | What the other DD owns |
|---|---|---|
| **BIL-02 Invoicing (Core)** | Uses Core's `InvoiceGenerationService.generate(ctx)` to invoke generators. Reuses `invoice` and `invoice_line_item` tables for the generated credit/debit note rows (via GEN-01). | Owns framework, tables, lifecycle. |
| **BIL-02-GEN-01 Invoice Generation** | Invokes `CreditNoteGenerator` and `DebitNoteGenerator` via Core's dispatch. Provides the `InvoiceGenerationContext` with adjustment-specific data. | Owns the generator classes and the note write logic. |
| **BIL-02-TAX-01 Tax Invoice & Gateway** | None directly. ADJ-01 cannot adjust tax invoices (TAX-01 owns signed cancellation). | Owns tax invoice lifecycle. |
| **BIL-02-READ-01 Invoice Read API** | None directly. Credit/debit note PDFs are retrieved via READ-01's unified invoice read API. | Owns PDF retrieval. |
| **BIL-02-STA-01 Status Management & Recovery** | None directly. Once applied, credit/debit notes follow the same lifecycle states as other invoices. | Owns lifecycle for invoices with due dates. |
| **BIL-01 Charging Engine** | Consumes `CreditNoteIssued` / `DebitNoteIssued` from Core's event publisher (emitted by GEN-01). Updates outstanding balance for POSTPAID; forwards to BIL-05 for PREPAID. | Owns balance updates and payment recording. |
| **BIL-05 Wallet and Topup** | For PREPAID: receives wallet credit/debit instructions and applies them. | Owns wallet mechanics. |
| **SUB-LM-01 Subscription Lifecycle Management** | Reads subscription state during validation (subscription must exist, not in blocking state). | Owns subscription master. |
| **PLM-CFG-02 Tax Configuration** | None directly. Generators (GEN-01) call PLM-CFG-02 if tax recomputation is needed on the adjustment. | Owns tax rules. |
| **Camunda Workflow Engine** | Deploys operator-specific BPMN files for approval workflows. Manages process instances. | Camunda is the runtime. |
| **CRM / Customer Service** | Called via Core's CustomerSnapshotService when GEN-01 captures customer snapshot on the note. | Owns live customer master. |

| Touched | What ADJ-01 owns | What the other DD owns |
|---|---|---|
| **DD_NOT-01 Notification Service** | Consumes `CreditNoteIssued` / `DebitNoteIssued` from Core; notifies customer. | Owns rendering + dispatch. |

## Dependencies

| Dependency | Owner |
|---|---|
| BIL-02 Invoicing (Core) | billing team (same wave) |
| BIL-02-GEN-01 Invoice Generation | billing team (same wave) |
| BIL-01 Charging Engine | billing team |
| BIL-05 Wallet and Topup | billing team |
| SUB-LM-01 Subscription Lifecycle Management | subscription team |
| SUB-WF-FRAMEWORK-01 Subscription Workflow Framework | subscription team |
| Camunda deployment | infrastructure team |
| DD_NOT-01 Notification Service | notification team |
| FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_CAMUNDA.md, FOUNDATION_AUTH.md | infrastructure team |
| Per-operator approval BPMN files | each operator's finance/operations team |
| Per-operator reason code catalog | each operator's finance team |
| Per-operator limits configuration | each operator's finance team |

## Operator-configurable capabilities (specific to ADJ-01)

| Capability | Where configured | Notes |
|---|---|---|
| Approval workflow | Operator-deployed Camunda BPMN referenced by `adjustment_approval_workflow_ref` | Zero-step (auto-approve), single-step, multi-step, threshold-based — any combination expressible in BPMN |
| Reason code catalog | `adjustment_reason_code` table (per-operator rows) | Operator defines their reason codes with display names per locale and limit-exempt flag |
| Limits per direction and currency | `adjustment_limits_config` | Max per request, max per customer per period, auto-approve threshold, single-step threshold |
| Insufficient-wallet-balance behavior for debit | Per-operator config | `FAIL_IMMEDIATELY` or `MARK_PENDING_FOR_NEXT_TOPUP` |
| AMOUNT-scope tax handling | Per-operator config | Operator chooses whether AMOUNT scope's amount is tax-inclusive (decompose into base + tax via PLM-CFG-02 Tax Configuration) or tax-exclusive (tax computed on top and added to the amount). LINE-scope and FULL-scope inherit tax structure from the parent invoice's lines, so this only applies to AMOUNT. |
| Adjustment notification template | Per-operator template in DD_NOT-01 Notification Service | Email/SMS/in-app layouts for credit/debit notes |

# B — Current TZ

Adjustments in TZ Sophix are handled ad-hoc without a structured workflow, approval gate, or audit trail. There is no first-class adjustment concept to describe or compare against.

# C — Gap & bridge

ADJ-01 is entirely new functionality as a structured workflow. Whatever ad-hoc correction patterns existed in TZ are superseded by the proposal → approval → application lifecycle described in section A.